



---

# Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

|                    |                              |
|--------------------|------------------------------|
| NIM                | 71251218                     |
| Nama Lengkap       | Agatha Sabda Alethea Prabawa |
| Minggu ke / Materi | 10 / Manajemen File          |

PROGRAM STUDI INFORMATIKA  
FAKULTAS TEKNOLOGI INFORMASI  
UNIVERSITAS KRISTEN DUTA WACANA  
YOGYAKARTA  
2026

## **BAGIAN 1: MATERI MINGGU INI (40%)**

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

### **Pengantar File**

Setiap program yang sedang berjalan memerlukan primary memory (memori utama) komputer sebagai tempat penyimpanan data sementara. Seluruh variabel, struktur data, dan informasi yang diolah oleh program tersimpan di sana selama program aktif. Namun begitu program dihentikan, semua data yang ada di primary memory ikut lenyap karena sifatnya yang volatile atau tidak dapat bertahan tanpa aliran listrik.

Kondisi ini tentu menjadi kendala apabila sebuah program membutuhkan data yang harus tetap tersedia meski program sudah ditutup. Untuk itulah dibutuhkan media penyimpanan yang bersifat permanen, yaitu secondary memory. Secondary memory mencakup perangkat seperti harddisk (HDD), solid-state drive (SSD), flash drive, dan sejenisnya. Berbeda dengan primary memory, data yang tersimpan di secondary memory tidak akan hilang meskipun komputer dimatikan.

Setiap file memiliki sejumlah atribut atau property yang melekat padanya, antara lain: nama file, ukuran (dalam satuan byte), lokasi penyimpanan di dalam harddisk, pemilik file (owner), hak akses (permission), serta tanggal pembuatan dan terakhir diakses. Atribut-atribut ini memungkinkan sistem operasi mengelola dan melindungi file secara terstruktur, serta memudahkan pengguna dalam menemukan dan mengidentifikasi file yang dibutuhkan.

### **Pengaksesan File**

Alur pengaksesan file pada Python mengikuti pola yang sederhana namun wajib dipatuhi agar file dapat diproses dengan benar dan sumber daya sistem tidak bocor.

Langkah-langkah pengaksesan file secara umum adalah sebagai berikut:

1. Menentukan nama file beserta jalurnya (path) yang ingin diakses.
2. Membuka file menggunakan perintah `open()` sehingga program memperoleh objek file (sering disebut file handle).
3. Melakukan operasi yang diinginkan terhadap file, seperti membaca (read) isinya atau menuliskan (write) data baru ke dalamnya.
4. Menutup file menggunakan perintah `close()` agar semua perubahan tersimpan dan sumber daya sistem dibebaskan.

Ketika fungsi `open()` dipanggil, Python tidak langsung memuat seluruh isi file ke dalam memori. Sebaliknya, fungsi ini mengembalikan sebuah objek file handle yang berperan sebagai "jembatan" antara program dengan file di disk. Melalui handle inilah seluruh operasi baca-tulis dilakukan. Berikut adalah contoh membuka file dan mencetak informasi handle-nya:

```
handle = open('mbox-short.txt')
print(handle)
```

# Output:

```
# <_io.TextIOWrapper name='mbox-short.txt' mode='r' encoding='UTF-8'>
```

Keluaran dari perintah `print(handle)` menampilkan tiga informasi penting: nama file yang dibuka, mode aksesnya (r berarti read atau baca saja), dan pengkodean karakter yang digunakan, yaitu UTF-8. Informasi ini mencerminkan status file handle yang siap digunakan oleh program.

Apabila nama file yang diberikan tidak ditemukan di lokasi yang ditentukan, Python akan langsung memunculkan pesan kesalahan berupa `FileNotFoundError`. Kesalahan ini perlu diantisipasi, terutama ketika nama file diperoleh dari input pengguna dengan menggunakan blok `try-except` agar program tidak berhenti mendadak.

# Contoh error saat file tidak ditemukan:

```
handle = open('file-tidak-ada.txt')
```

# Output:

```
# FileNotFoundError: [Errno 2] No such file or directory: 'file-tidak-ada.txt'
```

Secara praktis, file teks umumnya tersusun dari baris-baris string yang diakhiri karakter newline (`\n`). Model pembacaan yang paling umum dan dianjurkan adalah membaca file baris per

baris, bukan memuat seluruh isi file sekaligus sehingga program tetap efisien meski berhadapan dengan file berukuran sangat besar.

## Manipulasi File

### Membaca File Baris per Baris

Cara yang paling aman dan efisien untuk membaca sebuah file teks di Python adalah dengan memanfaatkan perulangan for yang secara otomatis mengambil satu baris pada setiap iterasinya. Metode ini bekerja dengan baik untuk file dengan ukuran berapa pun, karena hanya satu baris yang dimuat ke dalam memori pada satu waktu.

```
handle = open('mbox-short.txt')
count = 0
for line in handle:
    count = count + 1
print('Jumlah Baris:', count)
```

**# Output: Jumlah Baris: 1910**

Membuka seluruh isi file sekaligus menggunakan perintah `read()` memang memungkinkan, tetapi sangat berisiko jika ukuran file mencapai ratusan megabyte atau lebih. Memori komputer bisa habis terpakai dan program berpotensi hang atau crash. Teknik pembacaan baris per baris menjaga penggunaan memori tetap rendah dan program tetap responsif.

### Membaca Seluruh Isi File Sekaligus

Untuk file-file berukuran kecil, Python menyediakan metode `read()` yang memuat semua isi file menjadi satu string besar. Hal ini berguna apabila kita perlu mengakses konten file secara acak, misalnya mengambil karakter dari posisi tertentu menggunakan slicing. Namun perlu

diingat, metode ini sebaiknya hanya digunakan ketika ukuran file sudah dipastikan tidak terlalu besar.

```
handle = open('mbox-short.txt')
isi     = handle.read()

print('Ukuran file:', len(isi), 'byte')
print('16 karakter terakhir:', isi[-16:])
```

Fungsi `len()` yang diterapkan pada hasil `read()` akan mengembalikan jumlah total karakter dalam file. Jika diasumsikan setiap karakter setara dengan satu byte (berlaku untuk teks ASCII standar), maka nilai ini sekaligus menjadi perkiraan ukuran file dalam satuan byte. Notasi `isi[-16:]` memanfaatkan negative indexing dan slicing untuk mengambil 16 karakter terakhir dari isi file.

### Memfilter Baris Tertentu Saat Membaca File

Salah satu keunggulan membaca file baris per baris adalah kemampuan untuk menyaring hanya baris-baris yang relevan tanpa harus memproses seluruh isi file. Ini dilakukan dengan menggabungkan perulangan `for` dengan kondisi `if` di dalamnya. Sebagai contoh, program berikut hanya menampilkan baris yang dimulai dengan kata "Date:" (informasi tanggal) dari file log email:

```
handle = open('mbox-short.txt')
count  = 1
for line in handle:
    if line.startswith('Date:') and count <= 10:
        count += 1
        print(line.rstrip())

# Contoh output:
# Date: Sat, 5 Jan 2008 09:12:18 -0500
# Date: Fri, 4 Jan 2008 18:08:57 -0500
# ... dan seterusnya
```

Perhatikan penggunaan metode `rstrip()` pada contoh di atas. Setiap baris yang dibaca dari file sudah mengandung karakter newline (`\n`) di bagian akhirnya, ditambah lagi perintah `print()` secara bawaan juga menambahkan baris baru. Akibatnya, tanpa `rstrip()`, setiap baris output akan memiliki baris kosong ekstra di bawahnya. Memanggil `rstrip()` menghapus semua whitespace (termasuk newline) di bagian akhir string, sehingga keluaran menjadi lebih rapi.

Prinsip yang sama dapat diterapkan untuk berbagai keperluan lain: menampilkan baris yang mengandung kata kunci tertentu, menghitung jumlah baris yang memenuhi syarat, mengekstrak nilai numerik dari baris tertentu, dan sebagainya. Kombinasi pembacaan baris per baris dengan kondisi penyaringan adalah pola yang sangat umum dan powerful dalam pemrosesan file teks di Python.

## Penyimpanan File

Selain membaca, Python juga memungkinkan program untuk menulis data ke dalam sebuah file. Mekanismenya hampir identik dengan saat membuka file untuk dibaca, perbedaan utamanya hanya pada mode yang diberikan sebagai argumen kedua fungsi `open()`.

Berikut adalah tabel ringkas mode-mode yang tersedia saat membuka file:

| Kode             | Keterangan                                                                                                                                            |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| 'r'(read)        | Membuka file hanya untuk dibaca. Ini adalah mode default jika argumen mode tidak ditulis. Akan menghasilkan error jika file tidak ditemukan.          |
| 'w'(write)       | Membuka file untuk ditulis. Jika file sudah ada, seluruh isinya akan dihapus dan diganti. Jika file belum ada, file baru akan dibuat secara otomatis. |
| 'a'(append)      | Membuka file untuk ditambahkan konten di bagian akhir tanpa menghapus isi yang sudah ada. File baru akan dibuat jika belum ada.                       |
| 'r+'(read-write) | Membuka file untuk dibaca sekaligus ditulis. File harus sudah ada sebelumnya.                                                                         |

Untuk menulis data ke dalam file, digunakan metode `write()` pada file handle yang telah dibuka dengan mode 'w' atau 'a'. Argumen yang diterima `write()` adalah sebuah string — artinya jika ingin menyimpan nilai numerik, nilainya harus dikonversi ke string terlebih dahulu menggunakan `str()`. Setelah selesai menulis, file wajib ditutup menggunakan `close()` agar semua data yang tertampung di buffer benar-benar dituliskan ke disk dan sumber daya sistem dibebaskan.

Ada satu perbedaan mendasar antara `write()` dan `print()`, fungsi `print()` secara otomatis menambahkan karakter newline (`\n`) di akhir setiap pemanggilan, sedangkan `write()` tidak. Oleh karena itu, apabila ingin setiap baris tulisan berada pada baris tersendiri di file, karakter `\n` harus ditambahkan secara eksplisit di akhir setiap string yang ditulis.

## BAGIAN 2: LATIHAN MANDIRI (60%)

Link github: [https://github.com/gathasja/71251218\\_Agatha.git](https://github.com/gathasja/71251218_Agatha.git)

### SOAL 1

```
PrakAlPro-71251218 > minggu10 > soal01.py > ...
1  def bandingkanFile(file1,file2):
2      with open(file1, 'r') as f1, open(file2, 'r') as f2:
3          baris1 = f1.readlines()
4          baris2 = f2.readlines()
5
6          maxbaris = max(len(baris1), len(baris2))
7          for i in range(maxbaris):
8              b1 = baris1[i].rstrip() if i < len(baris1) else "tidak ada"
9              b2 = baris2[i].rstrip() if i < len(baris2) else "tidak ada"
10
11             if b1 != b2:
12                 print(f"baris {i+1}:")
13                 print(f"file 1: {b1}")
14                 print(f"file 2: {b2}")
15
16  bandingkanFile("soal1.txt", "soalsatu.txt")

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlPro> & C:/Users/lenovo/AppData/Local,
ro/PrakAlPro-71251218/minggu10/soal01.py"
baris 3:
file 1: kemarin tanggal 10 hujan badai
file 2: kemarin tanggal 7 hujan badai
baris 4:
file 1: coklat warna favoritku
file 2: merah warna favoritku
```

Penjelasan perlangkah soal01:

1. `def bandingkanFile(file1,file2):` -> untuk mendefinisikan fungsi bernama `bandingkanFile()` dengan 2 parameter `file1` dan `file2` (2 file yang dibandingkan nanti).
2. `with open(file1, 'r') as f1, open(file2, 'r') as f2:` -> Membuka **dua file sekaligus** dalam mode baca ('r'). `with` otomatis menutup file setelah selesai. `f1` dan `f2` adalah variabel penampung file.
3. `baris1 = f1.readlines()` -> Membaca semua baris dari file pertama (`f1`) dan menyimpannya sebagai list di variabel `baris1`.
4. `baris2 = f2.readlines()` -> Membaca semua baris dari file kedua (`f2`) dan menyimpannya sebagai list di variabel `baris2`.
5. `maxbaris = max(len(baris1), len(baris2))` -> Menghitung jumlah baris terbanyak di antara kedua file. Nilai ini digunakan untuk iterasi.
6. `for i in range(maxbaris):` -> Perulangan dari `i = 0` sampai `maxbaris-1`. `i` adalah indeks baris (dimulai dari 0).
7. `b1 = baris1[i].rstrip() if i < len(baris1) else "tidak ada"` -> Ambil baris dari file1:
  - Jika `i` masih dalam jangkauan `baris1` -> ambil baris ke-`i`, lalu hapus karakter newline dengan `rstrip()`
  - Jika `i` melebihi jumlah baris file1 -> isi dengan "tidak ada".
8. `b2 = baris2[i].rstrip() if i < len(baris2) else "tidak ada"` ->

Ambil baris dari file2:

- Jika i masih dalam jangkauan baris2 -> ambil baris ke-i, lalu hapus newline

- Jika i melebihi jumlah baris file2 -> isi dengan "tidak ada".

9. `if b1 != b2:` -> Bandingkan baris dari file1 dan file2. Jika berbeda, maka masuk ke blok if.

10. `print(f"baris {i+1}:")` -> Cetak nomor baris yang berbeda. i+1 karena baris dimulai dari 1 bukan dari 0.

11. `print(f"file 1: {b1}")` -> Cetak isi baris dari file pertama yang berbeda.

12. `print(f"file 2: {b2}")` -> Cetak isi baris dari file kedua yang berbeda.

13. `bandingkanFile("soal1.txt", "soalsatu.txt")` -> Memanggil fungsi `bandingkanFile` dengan argumen file "soal1.txt" dan "soalsatu.txt".

## SOAL 2

```
PrakAIPro-71251218 > minggu10 > soal02.py > ...
1  namaFile = "soal.txt"
2
3  print(f"nama file1: {namaFile}")
4  with open(namaFile, 'r', encoding="utf-8") as file:
5      for baris in file:
6          baris = baris.strip()
7          if not baris:
8              continue
9
10         if "||" in baris:
11             soal, jawabanBenar = baris.split("||")
12             soal = soal.strip()
13             jawabanBenar = jawabanBenar.strip()
14
15             print(f"\n{soal}")
16             inputUser = input("Jawab: ").strip()
17             if inputUser.lower() == jawabanBenar.lower():
18                 print("Jawaban benar!")
19             else:
20                 print("Jawaban salah!")
```

```
kAIPro-71251218/minggu10/soal02.py"
nama file1: soal.txt

1+1 =
Jawab: 2
Jawaban benar!

Bendera Indonesia?
Jawab: merah putih
Jawaban benar!

Kota gudeg adalah:
Jawab: yogya
Jawaban salah!

Komponen PC untuk penyimpanan file adalah...
Jawab: HARDDISK
Jawaban benar!

50 * 20 =
Jawab: 1000
Jawaban benar!
```

Penjelasan perlangkah soal02:

1. namaFile = "soal.txt" -> Menyimpan nama file yang akan dibaca ke dalam variabel namaFile.
2. print(f"nama file1: {namaFile}") -> Menampilkan nama file yang akan diproses.
3. with open(namaFile, 'r', encoding="utf-8") as file: -> Membuka file "soal.txt" dalam mode baca ('r').
  - encoding="utf-8" agar bisa membaca karakter khusus (seperti huruf dengan aksen).
  - with memastikan file ditutup otomatis setelah selesai.
  - file adalah variabel penampung file.
4. for baris in file: -> Perulangan untuk membaca file baris per baris (setiap iterasi mengambil satu baris dari file)
5. baris = baris.strip() -> Menghapus spasi di awal dan akhir baris, serta menghapus karakter newline (\n).
6. if not baris:
  - continue -> Mengecek apakah baris kosong setelah di-strip(), jika baris kosong, lewati baris ini dan lanjut ke baris berikutnya.
7. if "||" in baris: -> Mengecek apakah baris mengandung pemisah "||".
8. soal, jawabanBenar = baris.split("||") -> Memisahkan baris menjadi **dua bagian** menggunakan pemisah "||".

- Bagian kiri -> soal.

- Bagian kanan -> jawabanBenar.

9. `soal = soal.strip()` -> Menghapus spasi di awal/akhir teks soal.

10. `jawabanBenar = jawabanBenar.strip()` -> Menghapus spasi di awal/akhir teks jawaban benar.

11. `print(f"\n{soal}")` -> Menampilkan soal dengan baris baru di awal.

12. `inputUser = input("Jawab: ").strip()` -> Meminta input jawaban dari pengguna, lalu menghapus spasi di awal/akhir.

13. `if inputUser.lower() == jawabanBenar.lower():` -> Membandingkan jawaban user dengan jawaban benar (dalam huruf kecil semua, agar case-insensitive).

14. `print("Jawaban benar!")` -> Jika jawaban sama, tampilkan pesan "Jawaban benar!".

15. `else:`

`print("Jawaban salah!")` -> Jika jawaban tidak sama, tampilkan pesan "Jawaban salah!".